

ME570 Hwk1 Report

James U

September 2026

1 Question 1

1.1 Angle Between Segments

1.1.1 REPORT

First, segment angle computes the two vectors by subtracting vertex1 and vertex2 from vertex0 respectively. See Figure 1.

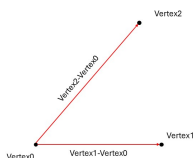


Figure 1: Segments angle compute the two vectors by subtracting the points from the origin.

Next, the two vectors are normalized to length 1. See Figure 2.

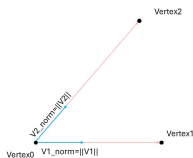


Figure 2: Vectors are normalized to length 1

The vectors are flattened into column vectors to facilitate taking the inner product for the vectors (which gives the angle value) and the cross product (which gives the sign of the angle). When the vectors are normalized, they are aligned with the parametric curve of the unit circle which can be defined as $\gamma(t) = (\cos t, \sin t)$. The vectors V_1 and V_2 identify with respective parameters of this parametric curve, namely that

$$V_1 = \gamma(t_1)$$

$$V_2 = \gamma(t_2)$$

θ can then be defined as the difference of the parameters t_2 and t_1 . This relationship allows us to recover θ by finding the inverrse tangent of the parametric values. Mathematically this can be written as:

$$\cos(t_2 - t_1) = \cos t_2 \cos t_1 + \sin t_2 \sin t_1 = V_1 \cdot V_2$$

$$\sin(t_2 - t_1) = \sin t_2 \cos t_1 - \cos t_2 \sin t_1 = V_1 \times V_2$$

$$\theta = \arctan(V_1 \times V_2, V_1 \cdot V_2)$$

The cross product gives the sign of the angle. See Figure 3. The information is then passed into the variable 'angle' and then returned.

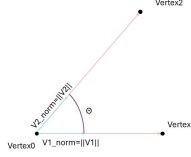


Figure 3: The dot product finds the angle magnitude, and the cross product finds the sign of the angle.

1.2 Intersection of Two Segments

1.2.1 REPORT

There are two ways that we can find whether the two segments intersect. The first is to treat the line segments as part of a parametric curve. The two segments $\overline{V_{A,1}, V_{A,2}}$ and $\overline{V_{B,1}, V_{B,2}}$ on $\mathbb{R}^2$ can be determined if they intersect by considering these two vectors as describing parametric curves which can be written as follows, where r , s , and t are scalar vectors:

$$L_1 : r = V_{A,1} + t(V_{A,2} - V_{A,1})$$

$$L_2 : r = V_{B,1} + s(V_{B,2} - V_{B,1})$$

We can solve this system by setting the equations equal to one another and solving for t and s . First, for simplicity, let $d_A = V_{A,2} - V_{A,1}$, $d_B = V_{B,2} - V_{B,1}$

$$V_{A,1} + t(d_A) = V_{B,1} + s(d_B)$$

This can be rearranged to give the displacement of the starting positions of vectors A and B . defined as:

$$w = V_{B,1} - V_{A,1} = t(d_A) - s(d_B)$$

The closed form solution to this problem then becomes:

$$t = \frac{w \times d_B}{d_A \times d_B}$$

And gives the point of intersection to be: $P = V_{A,1} + t(d_A)$.

If t and s exist, but are not in one or both of the line segments, then the vectors do not intersect. If t and s do not exist for $\overline{V_{A,1}, V_{A,2}}$ and $\overline{V_{B,1}, V_{B,2}}$, then the line segments do not meet.

While the first way is more generalizable, the second is less computationally complex as solving a system of equations to determine if overlap occurs. The second way to determine if the lines intersect requires the lines to be in the $\mathbb{R}^2$ plane. We consider two line segments first as the four points of their vertices, and we can measure the orientation of three points at a time using winding number and the orientation test. For the four vertices, there are four orientations to consider, as seen in the figure below. If the sign of orientation 1 and 2 are not equal AND if the sign of orientation 3 and 4 are not equal, then the lines intersect. Python code using this method can be seen in section 1.1.1.

If the orientation test does show that there is an intersection, that intersection can be found mathematically in the manner described above, where the parameterized line segments can be mathematically manipulated to find their intersection point.

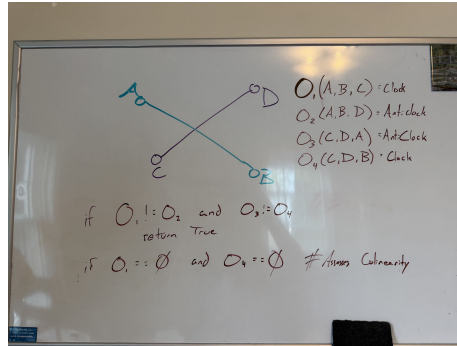


Figure 4: Orientation Test, using the sign of the orientation of the four vertices to determine if they intersect.

1.3 Segment Collision Test

1.3.1 REPORT

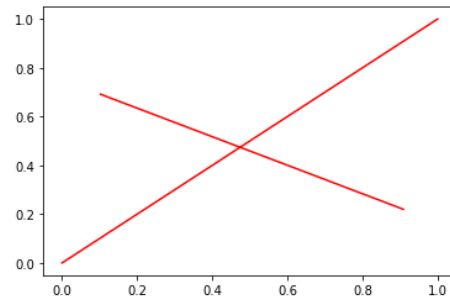


Figure 5: is_collision_test 1 - Intersecting line segments marked in red

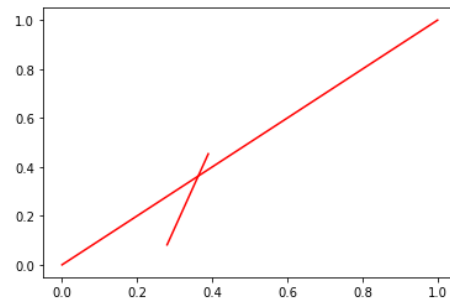


Figure 6: is_collision_test 2 - Intersecting line segments marked in red

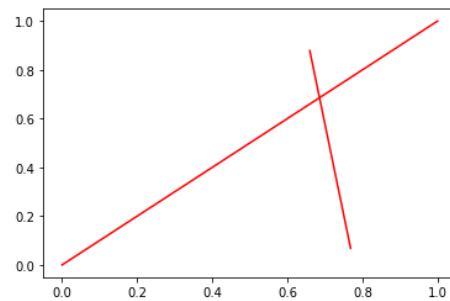


Figure 7: is_collision_test 3 - Intersecting line segments marked in red

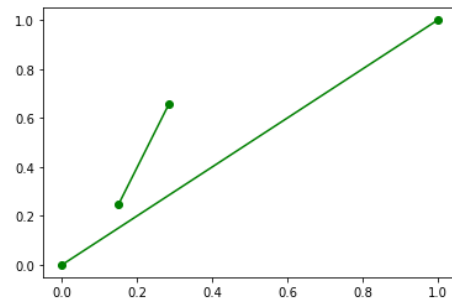


Figure 8: is_collision_test 4 - No intersecting line segments, marked in green

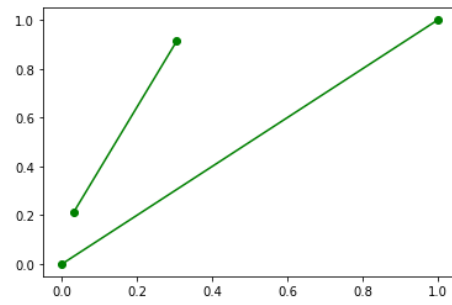


Figure 9: is_collision_test 5 - No intersecting line segments, marked in green

2 Question 2

2.1 Polygon Plotting

2.1.1 REPORT

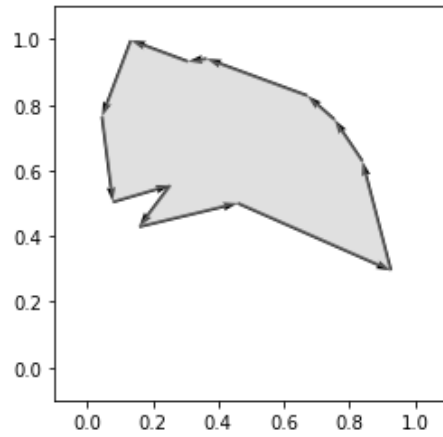


Figure 10: 12 sided polygon, counterclockwise, filled

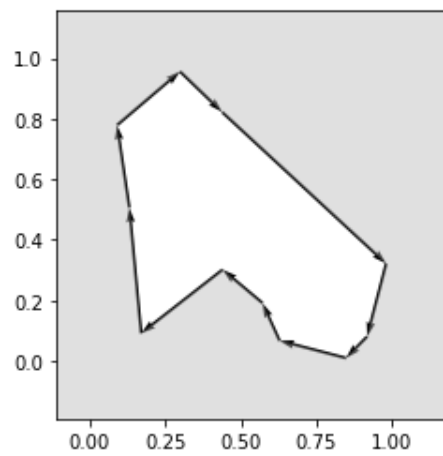


Figure 11: 11 sided polygon, clockwise, not filled

2.2 Winding Number and Collisions

2.2.1 REPORT

a) Given a point within the polygon, perhaps the centroid, we can use the winding number algorithm to determine whether a polygon is filled or hollow by the sign of the winding number's output. A filled polygon would move counterclockwise and output a $+1$. A hollow polygon would rotate clockwise and output a -1 . This relies on the problem setup that the polygon does not loop back onto itself. Thus we can determine a filled or hollow polygon based off of the sign of the winding number algorithm's output.

b) To test if a point is in collision with a polygon, you can compute the winding number from the point in question. If the winding number is zero, it means the point lies outside of the polygon; if the number is nonzero, then the point lies within the polygon. But this does not tell us whether the point is in collision with the polygon. By the explanation in part a, we can use the winding number tool to determine whether a polygon is filled or not filled. If the polygon is filled and the point's winding number is nonzero (i.e. within the polygon), then it is in collision with the polygon. Likewise, if the polygon is not filled and the point's winding number is zero, then the point is in collision with the exterior portion of the polygon which counts as 'filled.' The opposite for both of these statements is also true to define when points are not in collision with the filled region, namely that a filled polygon and a point with a winding number of zero are not in collision, and that a not filled polygon with a nonzero winding number are not in collision.

2.3 Is Collision?

2.3.1 REPORT

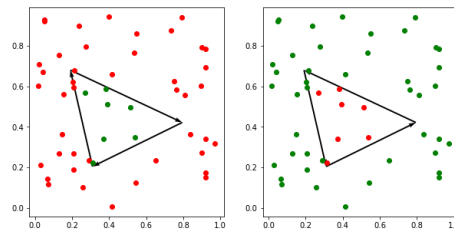


Figure 12: is_collision test 1, collisions marked in red, no collisions marked in green

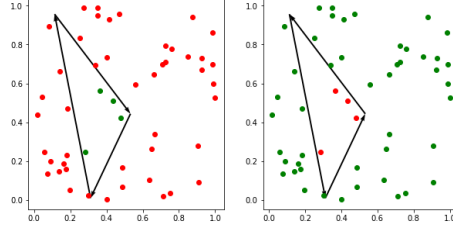


Figure 13: is_collision test 2, collisions marked in red, no collisions marked in green

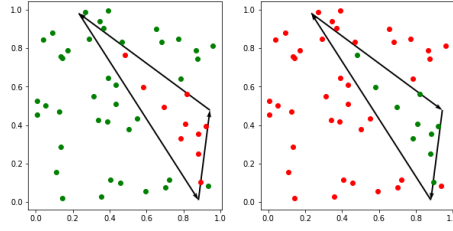


Figure 14: is_collision test 3, collisions marked in red, no collisions marked in green

3 Question 3

3.1 Explain Quantities

a) $\sum_i p_i = 1$. The sum of all (i.e. quantity N) probabilities in the state space Ω sums to 1. This partitions the state space Ω and represents the sum of all possible outcomes of an event.

b) $\sum_i x_i p_i = \mu$. The sum of each entry x_i multiplied by the scalar probability of its respective occurrence, p_i , gives the mean of the PMF, here represented by μ . It is important to note that since $x_i \in \mathbb{R}^2$, the mean $\mu \in \mathbb{R}^2$ as well.

c)

$$\sum_i (x_i - \sum_{i'} x_{i'} p_{i'}) (x_i - \sum_{i'} x_{i'} p_{i'})^T p_i = \dots$$

From part b, we know that $\sum_i x_i p_i = \mu$. From this we can also say that $\sum_i x_i p_i = \mathbb{E}[x_i]$. Therefore, we can rewrite the above to be:

$$\dots = \mathbb{E}[(x_i - \mathbb{E}[x_i])(x_i - \mathbb{E}[x_i])^T] = \text{Cov}[x_i, x_i] = \text{Var}[x_i]$$

Thus we can write that $\sum_i (x_i - \sum_{i'} x_{i'} p_{i'}) (x_i - \sum_{i'} x_{i'} p_{i'})^T p_i = \text{Var}[x_i]$.

Information for part c was sourced from Wikipedia, Covariance Matrix and Khan Academy's video on variance and standard deviation of discrete random variables.